

uAPI Project Documentation

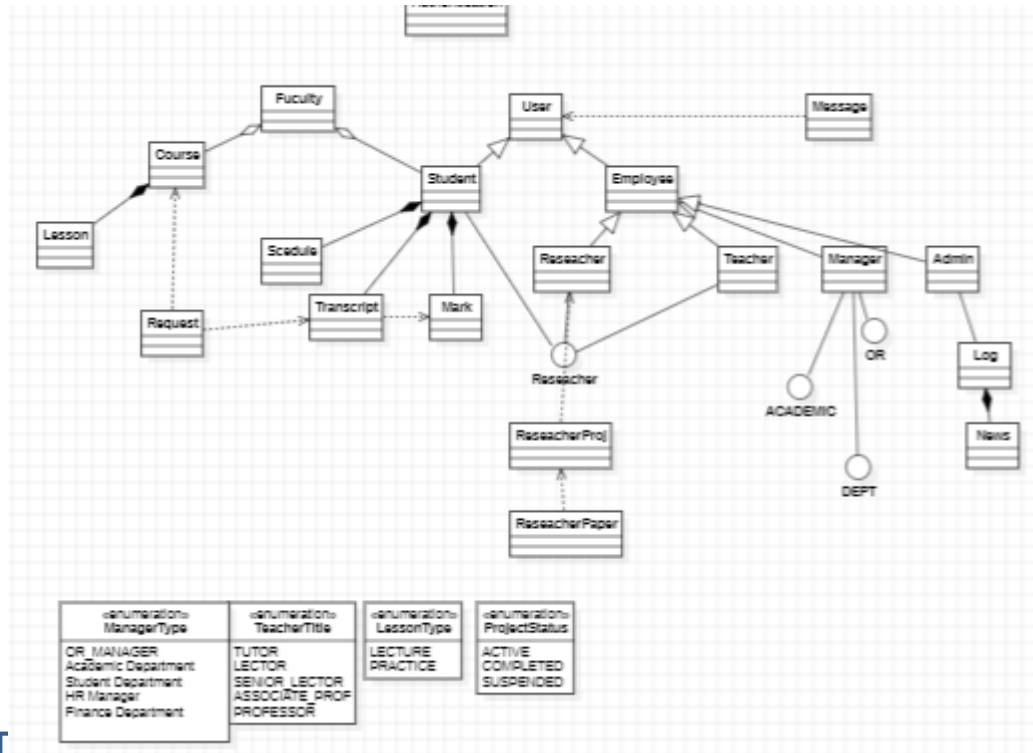
University API Management System

Date: 08.05.2026

Content

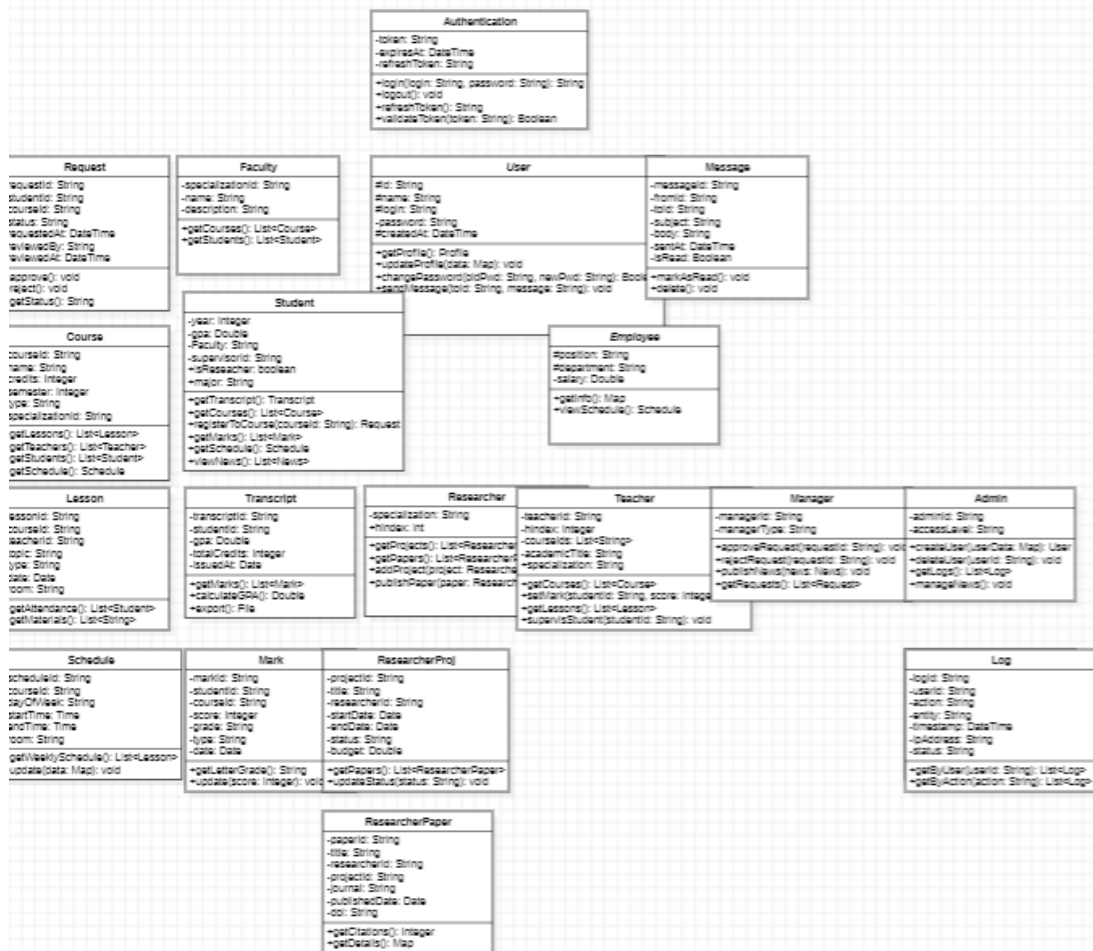
- 1. UML DIAGRAM
- 0. Architecture Description
- 2. Classes
- 3. Interfaces
- 4. Enums
- 5. Code Snippets
- 6. Challenges and challenges
- 7. Project Management

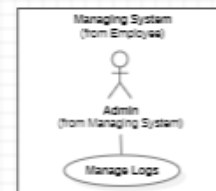
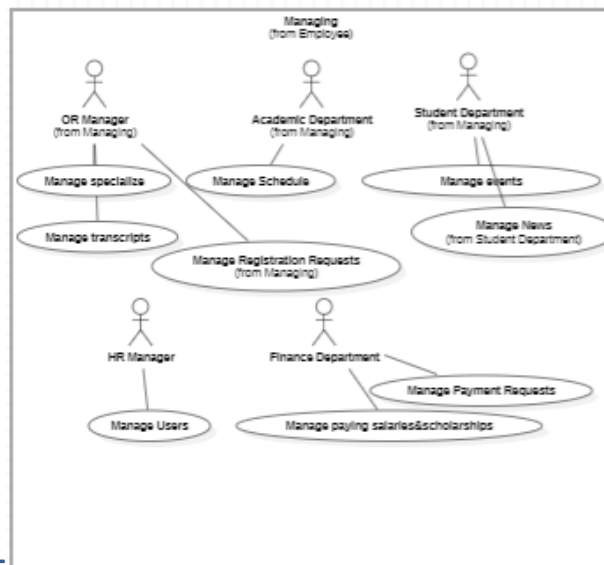
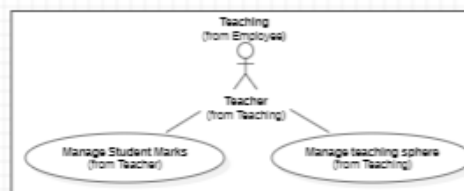
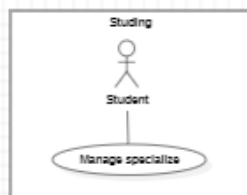
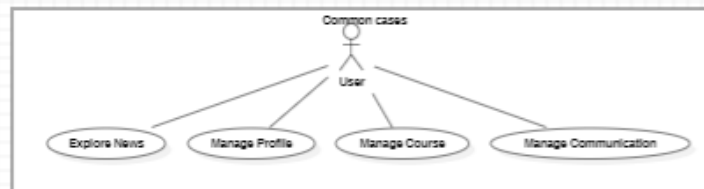
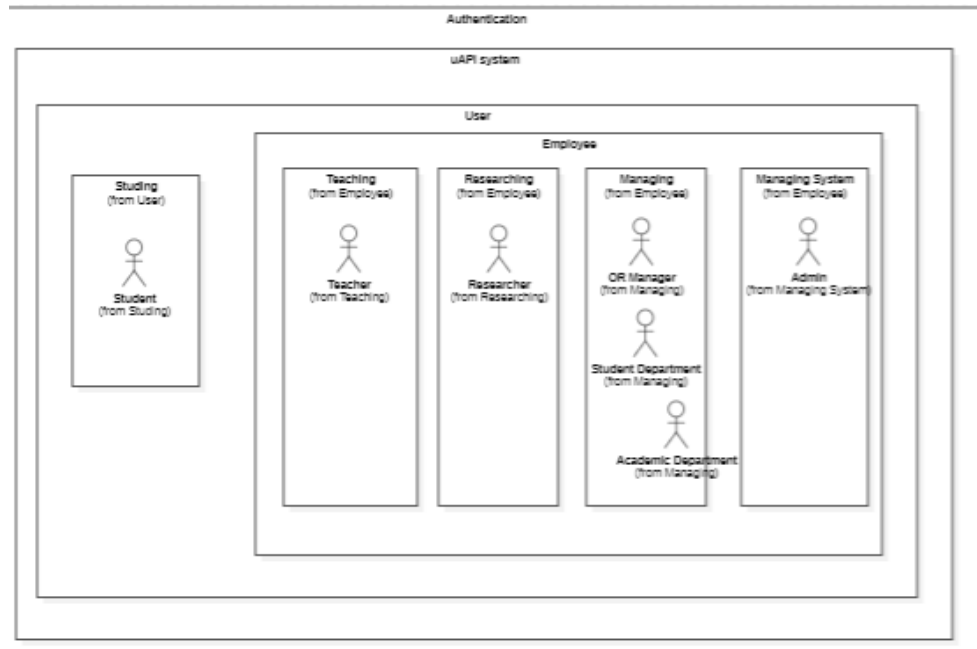
UML DIAGRAM



OBJECT

CLASS





USECASE

TELEGRAMM CHAT

OOP Proj

4 members

Pinned message

<https://github.com/ZhagiparIIDidar/uAPI.git>

April 23

Всем привет 11:35 ✓✓

Изучите эти 4 паттерна. Когда будете писать код используйте
1.Singleton База данных / Data Storage
2.Observer Уведомления (новости, сообщения)
3.Factory Создание пользователей по типу
4.Decorator Дополнительные роли (например Researcher
поверх Teacher) 11:36 ✓✓

Скоро я вам отправлю uml диаграмму 11:36 ✓✓

Да и диаграмму изучите чтобы понять что там показано 11:36 ✓✓

Если что то не понятно спрашивайте 11:37 ✓✓



И имена напишите чтобы я мог поделить ответственность
edited 11:38 ✓✓



Kozha
Калауов Кажымухан 11:52



Ayala
Кенжебай Аяла 12:00



Кеңесов Мирас 13:48

1. Architecture description

Проект The uAPI (University API) project is a Java-based university management system that uses a multi-level architecture divided into:

Level	Description
Presentation Layer	Menu classes for user interaction
Domain Layer	Data Model classes (Course, Faculty, Lesson, Mark)
Service Layer	Management interfaces (Managers, Request handlers)
Data Access Layer	DatabaseManager, UserDao for working with SQLite databases

2. Classes

2.1 Abstract classes

User (kz.uapi.abs_class.User)

Abstract base class for all users of the system. Serves as the basis for the user hierarchy.

Поле	Тип
login	String
password	String
role	String
firstName	String
lastName	String
createdAt	LocalDateTime

Sample code:

```
public abstract class User {
    private String login;
    private String password;
    private String role;
    // ... getters and setters
}
```

Employee (kz.uapi.abs_class.Employee)

Abstract class for all employees (teachers, administrators). Inherits User and adds fields for employees.

Field	Type
department	String

salary	double
--------	--------

2.2 Specific classes

Student (kz.uapi.users.Student)

Inherits: User

Represents the student in the system. Contains information about your GPA, course, and specialty.

Поле	Тип
gpa	double
year	int
major	String
credits	int
isResearcher	boolean
supervisorLogin	String

Teacher (kz.uapi.users.Employee.Teacher)

Inherits: Employee, implements Researcher

Represents the teacher. Can be a researcher (professor).

Поле	Тип
title	TeacherTitle
rating	double
hIndex	int
specializations	List<String>

Course (kz.uapi.domain.academic.Course)

Inherits: Object

Presents a course with information about the teacher and students.

Поле	Тип
courseId	int
name	String
description	String
credits	int
teacherLogin	String
maxStudents	int
facultyId	int

Faculty (kz.uapi.domain.academic.Faculty)

Inherits: Object

Represents the university faculty.

Field	Type
facultyId	int
name	String

3. Interfaces

3.1 Manager Interfaces

AcademicManager

Managing the learning process and schedule

- Methods:
 - void manageSchedule()

FinanceManager

Financial, salary and scholarship management

- Methods:
 - void manageSalaries()
 - void manageScholarships()

HrManager

Managing employees

- Methods:
 - void addUser()
 - void dropUser()

OrManager

Managing organizational issues

- Methods:
 - void approveRegistration()

StudentDepartmentManager

Managing the Student Department

- Methods:
 - void manageDepartment()

3.2 Request/Researcher Interfaces

Researcher

Interface for researchers

- Methods:
 - `int getHIndex()`
 - `List<String> getSpecializations()`

RegistrationRequest

Interface for registration requests

ResearchRequest

Interface for scientific queries

ScheduleRequest

Interface for schedule requests

4. Enums

TeacherTitle

Teacher titles

Value
of TUTOR (Teacher)
LECTURER
SENIOR_LECTOR (Senior Lecturer)
PROFESSOR

LessonType

Types of classes

Meaning
of LECTURE (Lecture)
PRACTICE (Practice)

RequestStatus

Request statuses

Meaning
PENDING value
APPROVED
REJECTED

RequestType

Request types

Value
of COURSE_REGISTRATION
RESEARCH_PROPOSAL
SCHEDULE_CHANGE

ManagerType

Types of managers

Meaning
ACADEMIC
FINANCE

HR
ORGANIZATIONAL Value

5. Code Snippets

5.1 Creating a student

```
Student student = new Student(
    "student001",          // login
    "password123",        // password
    "Иван",                // firstName
    "Иванов",              // lastName
    LocalDateTime.now(),  // createdAt
    3.8,                   // gpa
    2,                     // year
    "Computer Science",    // major
    60,                    // credits
    false,                 // isResearcher
    null                   // supervisorLogin
);
```

5.2 Creating a course

```
Course course = new Course(
    1001,                  // courseId
    "Advanced Java",       // name
    "Learn Java OOP",      // description
    4,                     // credits
    "teacher001",          // teacherLogin
    30,                    // maxStudents
    1                      // facultyId
);
```

5.3 Working with the teacher

```
Teacher teacher = new Teacher(
    "teacher001",
    "pwd456",
    "Петр",
    "Петров",
    LocalDateTime.now(),
    "Computer Science",
    50000.0,
    TeacherTitle.PROFESSOR, // title
    4.7,                    // rating
    25,                     // hIndex
    Arrays.asList("AI", "ML", "Java") // specializations
);

if (teacher.isResearcher()) {
    System.out.println("H-Index: " + teacher.getHIndex());
}
```

6. Challenges and challenges

1. Managing a complex class hierarchy

- Problem: Problem: Deep hierarchy User - > Employee → Teacher complicates management.
- Solution: Use interfaces (Researcher, AcademicManager) to share responsibilities.

2. Data consistency

- Problem: Problem: Synchronization between SQL databases and object models.
- Solution: Using the DAO pattern (UserDao, DatabaseManager) for data access abstraction.

3. Scalability

- Problem: Problem: When adding new user types, you need to change the existing code.
- Solution: Factory pattern for creating objects, extensible interface system.

4. Managing access rights

- Problem: Problem: Different users have different rights and functions.
- Solution: Role-based access control (RBAC) using the role field and interfaces.

5. Exception handling

- Problem: Problem: Insufficient error handling when working with the database and data validation.
- Solution: Add try-catch blocks, custom exceptions, and input data validation.

7. Project Management

7.1 Project structure

```
uAPI/
├── pom.xml (Maven configuration)
├── src/
│   ├── main/java/kz/uapi/
│   │   ├── abs_class_class/ (Abstract classes)
│   │   ├── users/          (User classes)
│   │   ├── domain/         (Data Models)
│   │   ├── --- database/ (DAO layer)
│   │   ├── enums/         (Enums)
│   │   ├── interfaces/    (Interfaces)
│   │   ├── menu/          (UI layer)
│   │   └── App.java (Entry Point)
│   └── resources/
├── target/ (Compiled code)
└── javadoc/ (HTML documentation)
```

7.2 Project statistics

Metric	Value
of Classes	25 +
Interfaces	9
Enums	5
Packages	8
Lines of code (approx)	2000+
Documented classes	8 documented classes (with Javadoc)

7.3 Development status

- Core Architecture: Completed ✓
- Database Layer: Completed ✓
- User Management: Completed ✓
- Academic Management: In development
- Research Management: In development
- UI / Menu System: Under development
- Testing: Required

- Documentation: In progress ✓

7.4 Development plan

- Phase 1: Complete the main classes and interfaces
- Phase 2: Implement full DB integration
- Phase 3: Develop a Web Interface (Spring Boot MVC)
- Phase 4: Add a REST API (Spring Data REST)
- Phase 5: Write Unit Tests (JUnit 5)
- Phase 6: Configure CI / CD (GitHub Actions)

7.5 Technology stack

Component	Technology
Java Programming	Language 25
Build Tool	Maven 3.9+
SQLite Database	3.45
IDE	VS Code / IntelliJ
Documentation	IDEA Javadoc Documentation

7.6 Tasks for the team

Task	Priority	Status
Complete implementation of all Manager Interfaces	High	In progress
Write unittests for the DAO layer	High	Need
to Improve Error handling	Medium	Need
to Optimize SQL queries	Medium	Need
to Add Logging	Low	Need
to Create API documentation	Medium	In progress

Conclusion

The uAPI project uAPI is a well-structured university management system in Java with a clear architecture and division of responsibility. The main focus is on using interfaces and abstract classes to ensure flexibility and extensibility of the system. Further development should focus on completing the implementation of business logic, writing tests, and optimizing performance.

Document created: 08.05.2026 21: 38: 51

Version: 1.0